

# Containerization Using Docker

From Basic to Industry Level – A practical guide for Machine Learning & AI projects

---

**Dr. Anilkumar Vishwanath Brahamne**

Computer Engineering Department, Sanjivani College of Engineering



# The "Works on My Machine" Problem

## Your ML Project Needs

- Python 3.12
- NumPy, Pandas, Scikit-learn
- Flask & TensorFlow (specific version)
- Source code & config files

## ✗ What Goes Wrong

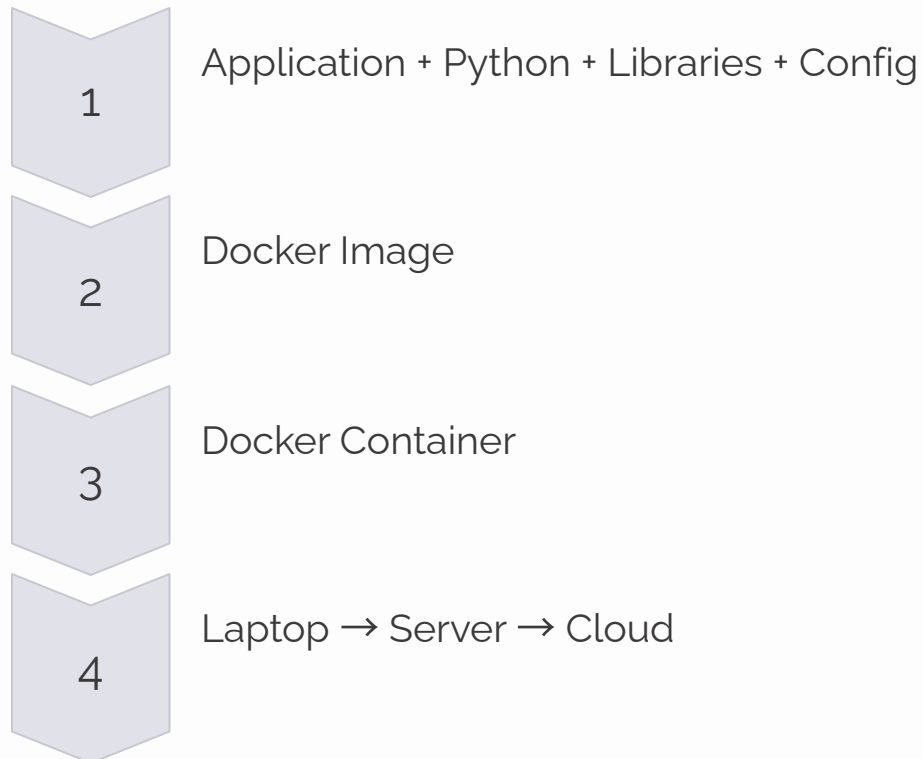
When you share the project or deploy to a server, the other machine may have a different Python version, missing libraries, a different OS, or different environment variables — causing failures.

- ❏ **Containerization solves this** by packaging the application together with everything it needs into a portable unit called a **container**.

*Dr. Anilkumar Vishwanath Brahamne | Computer Engineering Department, Sanjivani College of Engineering*

# What is Docker? A Simple Analogy

Think of a **lunch box**. Instead of carrying rice, dal, vegetables, spoon, plate, and water separately, you pack everything into one box. Docker does the same for your application.



**Docker** is a platform used to create, package, distribute, and run applications inside containers so they behave consistently across different machines.

*Dr. Anilkumar Vishwanath Brahamne | Computer Engineering Department, Sanjivani  
College of Engineering*



# Key Docker Terminology



## Dockerfile

A text file with instructions for building a Docker image.



## Docker Image

A packaged blueprint/template of your application – read-only.



## Docker Container

A running instance of an image – the live application.



## Docker Registry

A place to store and share Docker images (e.g., Docker Hub).



## Docker Engine

The software that builds and runs containers on the host machine.

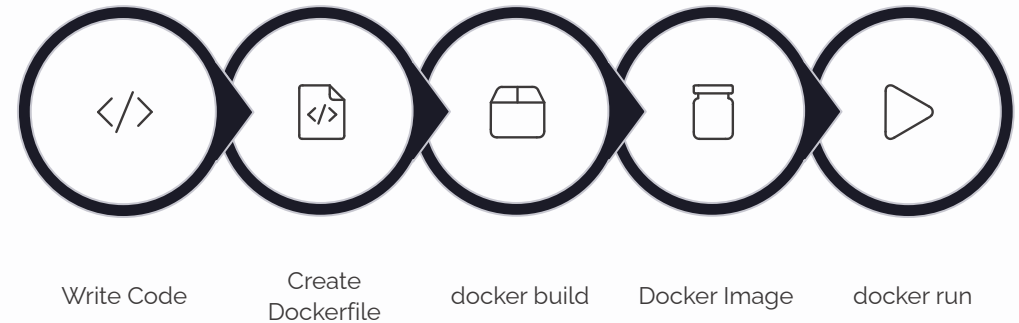
# Dockerfile & Docker Workflow

## Sample Dockerfile (ML + Flask)

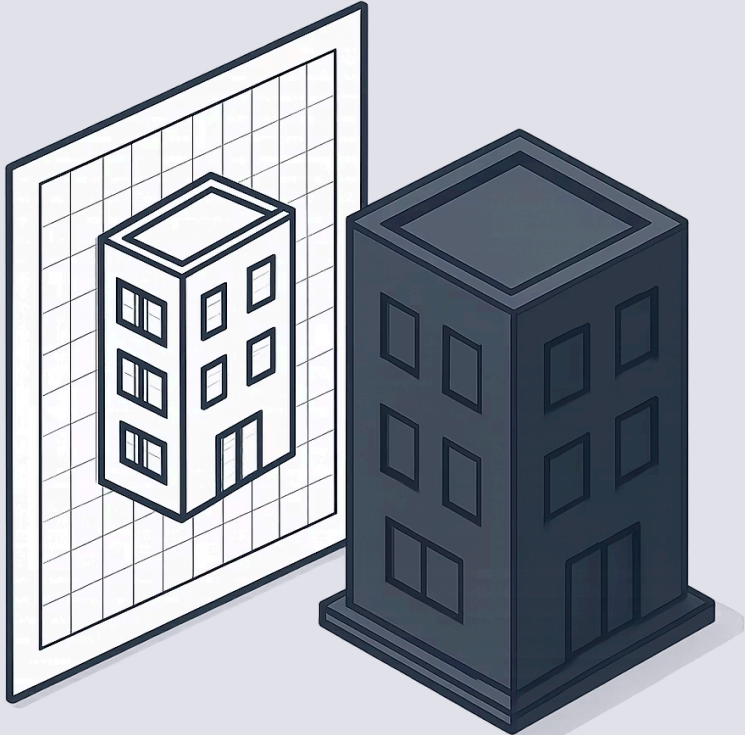
```
FROM python:3.12
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

**FROM** sets the base image. **WORKDIR** sets the directory. **COPY** brings in files. **RUN** installs libraries. **EXPOSE** declares the port. **CMD** starts the app.

## Complete Workflow



# Image vs. Container — Key Interview Concept



## Docker Image

- Blueprint / template
- Read-only package
- Used to create containers
- Does not run by itself
- One image → many containers

## Docker Container

- Running instance of an image
- Live execution environment
- Created from an image
- Runs the application
- Each container is independent

**Easy analogy:** Classroom blueprint = Image. Actual classroom = Container.

*Dr. Anilkumar Vishwanath Brahamne | Computer Engineering Department, Sanjivani  
College of Engineering*

# ML Project: House Price Prediction with Docker

## Project Structure

```
house-price-prediction/  
├── app.py  
├── model.pkl  
├── requirements.txt  
├── Dockerfile  
└── templates/  
    └── index.html
```

## requirements.txt

```
flask  
numpy  
pandas  
scikit-learn  
joblib
```

## Build & Run Commands

```
docker build -t ml-api .
```

```
docker run -p 5000:5000 ml-api
```

The **-p 5000:5000** flag maps host port 5000 to container port 5000, making the Flask ML API accessible from the browser or Postman.

Docker packages all of:

- Python runtime + Flask
- NumPy, Pandas, Scikit-learn, Joblib
- Trained model (model.pkl) + app code

# Why Industry Uses Docker



## Portability

Move seamlessly: Laptop → Server → Cloud.



## Isolation

Apps run in separate containers without conflict.



## Faster Deployment

Deploy a container instead of manual installs.



## Consistency

Same environment everywhere — no surprises.



## Scalability

Run multiple containers behind a load balancer.



## Dependency Management

All dependencies are packaged with the application.

*Dr. Anilkumar Vishwanath Brahamne / Computer Engineering Department, Sanjivani College of Engineering*

# Docker Compose, Kubernetes & VM vs. Container

## VM vs. Docker Container

### Virtual Machine

Each VM runs a full guest OS — heavy and slow to start.

### Docker Container

Shares host OS kernel — lightweight and fast.

## Docker Compose

Manages multiple containers (Frontend, Backend, Database, ML API) using a single `compose.yaml` file. Start all services with `docker compose up`.

## Docker vs. Kubernetes

### Docker

Build, package, run, and distribute containers.

### Kubernetes

Manage, scale, monitor, restart, and orchestrate containers in production.

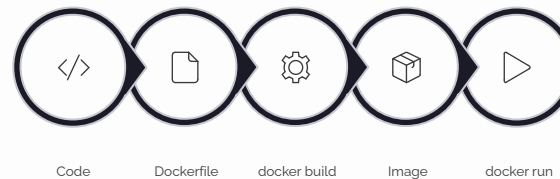
Docker creates containers. Kubernetes manages those containers at scale across cloud environments like AWS, Azure, and GCP.

# The Most Important Concept & Student Project

Docker takes your application + dependencies, packages them into an image, and runs that image as a container so the application can be deployed consistently across different environments.



## Remember This Sequence



## Student Practical Project

### Deploy an ML Model using Flask & Docker

1. Train ML model on a dataset
2. Save model: `joblib.dump(model, "model.pkl")`
3. Create Flask API with POST `/predict`
4. Write `requirements.txt` & `Dockerfile`
5. Build: `docker build -t ml-api .`
6. Run: `docker run -p 5000:5000 ml-api`
7. Test API from browser or Postman

Next topic: **Dockerizing a Flask ML model step-by-step** — from `model.pkl` to live API.